

Data Structure

Level 1

Doubly Linked List Operation



Data Structure

Level 1

Map

Definition

A `std::map` is an STL container that stores data as key-value pairs.

- ◆ Each key is unique.
- ◆ Elements are automatically sorted by key in ascending order by default.
- ◆ Each key is associated with one value.

Explanation

◆ A `map` is useful when you want to find a value using a key.

■ Main characteristics:

▨ Key-Value Pairs → Each element contains a key and a value

▨ Unique Keys → Each key can appear only once

▨ Sorted Keys → Keys are automatically sorted

▨ Efficient Lookup → Searching by key takes $O(\log n)$

▨ Dynamic Size → The map grows and shrinks automatically

▨ Iterator Support → You can traverse all elements

◆ Delete the last node using delete.

Procedures (Map)

1. Declare a Map

■ Use:

```
map<string, int> studentGrades;
```

➡ Result: Creates a map where:

▨ Key = `string`

▨ Value = `int`

2. Insert Elements

■ Add key-value pairs:

```
studentGrades["Ali"] = 77;
```

```
studentGrades["Ahmed"] = 85;
```

```
studentGrades["Fadi"] = 95;
```

➡ Result:

Ahmed → 85

Ali → 77

Fadi → 95

✦ Notice that the elements are sorted alphabetically by key, not by the order they were inserted.

Procedures (Map)

3 Access a Value

- Use the key:

```
studentGrades["Ali"]
```

- Result:

77

4 Find an Element

- Use the `find()` function:

```
studentGrades.find("Fadi")
```

- If the key exists:

- `find()` returns an iterator to that element

- If the key does not exist:

- `find()` returns `end()`

- Example:

```
if (studentGrades.find("Fadi") != studentGrades.end()) {  
    cout << "Found";  
}
```

- Result:

Found

Procedures (Map)

Iterate Through the Map

Use a range-based `for` loop:

```
for (const auto& pair : studentGrades) {  
    cout << pair.first << " " << pair.second;  
}
```

◆ `pair.first` → Key

◆ `pair.second` → Value

➡ Result:

Ahmed 85

Ali 77

Fadi 95

💡 Example

Suppose we have:

```
map<string, int> studentGrades;  
studentGrades["Ali"] = 77;  
studentGrades["Ahmed"] = 85;  
studentGrades["Fadi"] = 95;
```

The map stores:

Key	Value
-----	-------

Ahmed → 85

Ali → 77

Fadi → 95

🔍 Searching for `"Fadi"`:

```
studentGrades.find("Fadi")
```

➡ Result:

Fadi → 95

🔍 Searching for `"Omar"`:

```
studentGrades.find("Omar")
```

➡ Result:

Grade not found

Note

◆ `std::map` is commonly used when you need to associate one piece of information with another.

💡 Examples:

▣ Student name → Grade

▣ ID → Employee name

▣ Country → Capital

▣ Product ID → Price

◆ `std::map` is usually implemented using a balanced search tree, commonly a red-black tree.

Comparison

`std::map`

- ✓ Stores key-value pairs
- ✓ Keys are unique
- ✓ Keys are sorted
- ✓ Search: $O(\log n)$
- ✓ Insert: $O(\log n)$
- ✓ Delete: $O(\log n)$

`std::vector`

- ✓ Stores elements sequentially
- ✓ Allows duplicate values
- ✓ Elements are not automatically sorted
- ✓ Access by index: $O(1)$
- ✓ Searching: $O(n)$

⊘ Warning

⚠ Keys in a ``std::map`` must be unique

⚠ If you use:

```
studentGrades["Ali"] = 90;
```

when ``"Ali"`` already exists, the old value ``77`` will be replaced with ``90``.

➡ Result:

Ali → 90

✦ Also, using ``map[key]`` can create a new element if the key does not already exist.

✦ Summary

- ✓ `std::map` stores key-value pairs
- ✓ Each key must be unique
- ✓ Keys are automatically sorted
- ✓ `find()` is used to search for a key
- ✓ `pair.first` represents the key
- ✓ `pair.second` represents the value
- ✓ Search, insertion, and deletion are $O(\log n)$

Thank you 😊